

Modelagem MongoDB – Aggregations e Finalização

FATEC Jacareí · Dia 4: Aggregations e Finalização · 25/05/26

1. Contexto

Com o banco **leads_db** populado (seed.js) e as consultas operacionais definidas (consultas.js), o Dia 4 implementa os pipelines de aggregation para indicadores gerenciais do sistema de gestão de leads da **1000 Valle Multimarcas**. Os cinco pipelines estão no arquivo **aggregations.ts**.

2. Operadores Utilizados

Operador	Função
\$match	Filtra documentos antes de processar (equivalente ao WHERE).
\$group	Agrupa documentos por um campo e aplica acumuladores (\$sum, \$avg...).
\$sort	Ordena o resultado do pipeline.
\$project	Seleciona/renomeia campos na saída.
\$lookup	Faz join entre coleções (referencing).
\$unwind	Desconstrói um array — necessário após \$lookup que retorna array.
\$cond	Expressão condicional (if/else) dentro de acumuladores.
\$divide / \$multiply	Operações aritméticas — usadas no cálculo da taxa de conversão.

3. Pipelines (aggregations.ts)

3.1 Leads por Canal (channel)

Conta quantos leads vieram de cada canal de captação. Útil para saber quais canais geram mais oportunidades.

Pipeline:

```
db.leads.aggregate([
  { $match: {} },
  { $group: { _id: "$channel", total: { $sum: 1 } } },
  { $sort: { total: -1 } },
  { $project: { origem: "$_id", total: 1, _id: 0 } }
])
```

Saída esperada:

```
{ origem: "whatsapp", total: 4 } { origem: "instagram", total: 3 } ...
```

3.2 Leads por Status (via negotiations)

Agrupa as negociações pelo campo status. Mostra quantas estão abertas e quantas foram encerradas.

Pipeline:

```
db.negotiations.aggregate([
  { $match: {} },
```

```
{ $group: { _id: "$status", total: { $sum: 1 } } },
{ $sort: { total: -1 } },
{ $project: { status: "$_id", total: 1, _id: 0 } }
])
```

Saída esperada:

```
{ status: "aberta", total: 7 } { status: "encerrada", total: 3 }
```

3.3 Taxa de Conversão

Calcula o percentual de negociações com status "encerrada" sobre o total. Usa \$cond para contar condicionalmente e aritmética para calcular o percentual.

Pipeline:

```
db.negotiations.aggregate([
  { $match: {} },
  { $group: {
    _id: null,
    total: { $sum: 1 },
    encerradas: { $sum: { $cond: [{ $eq: ["$status", "encerrada"] }, 1, 0] } }
  } },
  { $project: {
    total: 1,
    encerradas: 1,
    taxaConversao: { $multiply: [{ $divide: ["$encerradas", "$total"] }, 100] }
  } }
])
```

Saída esperada:

```
{ total: 10, encerradas: 3, taxaConversao: 30 }
```

3.4 Leads por Atendente (\$lookup)

Agrupar os leads por attendantId e usar \$lookup para trazer o nome do usuário da coleção users. \$unwind desfaz o array retornado pelo \$lookup para acessar os campos do documento.

Pipeline:

```
db.leads.aggregate([
  { $match: {} },
  { $group: { _id: "$attendantId", total: { $sum: 1 } } },
  { $lookup: {
    from: "users",
    localField: "_id",
    foreignField: "_id",
    as: "user"
  } },
  { $unwind: "$user" },
  { $sort: { total: -1 } },
  { $project: { atendente: "$user.name", total: 1, _id: 0 } }
])
```

Saída esperada:

```
{ atendente: "Ana", total: 4 } { atendente: "Bruno", total: 3 } ...
```

3.5 Leads por Importância

Agrupar as negociações pelo campo importance (quente / morno / frio). Permite priorizar o funil comercial.

Pipeline:

```
db.negotiations.aggregate([
  { $match: {} },
  { $group: { _id: "$importance", total: { $sum: 1 } } },
  { $sort: { total: -1 } },
  { $project: { importancia: "$_id", total: 1, _id: 0 } }
])
```

Saída esperada:

```
{ importancia: "quente", total: 3 } { importancia: "morno", total: 3 } {
importancia: "frio", total: 4 }
```

4. Resumo do Projeto (leads_db)

Visão consolidada do que foi implementado ao longo dos 4 dias:

Dia	Entrega	Arquivos
1 – 04/05	Leitura do problema e definição das coleções	—
2 – 11/05	Modelagem: embedding vs referencing	—
3 – 18/05	Inserção de dados e consultas operacionais	seed.js · consultas.js · coleções.js
4 – 25/05	Aggregations e finalização	aggregations.ts

Estrutura final do banco:

Coleção	Volume	Campos principais
users	5 docs	admins, gerentes, atendentes
stores	3 docs	lojas: Centro, Norte, Sul
customers	5 docs	clientes com createdBy → users
leads	10 docs	channel, customerId, storeId, attendantId
negotiations	10 docs	importance, status, currentStage, history[] (embedding)
logs	10 docs	userId, action, entity, createdAt
teams	—	reservada

5. Conclusão

O projeto implementou um banco MongoDB completo para gestão de leads, cobrindo modelagem, inserção de dados e consultas em quatro entregas. A combinação de **referencing** para entidades independentes (users, stores, customers) e **embedding** para o histórico de negociações permitiu consultas performáticas sem joins desnecessários. Os cinco pipelines de aggregation entregam os indicadores gerenciais do funil comercial da 1000 Valle Multimarcas diretamente do MongoDB.

- Banco: leads_db · 7 coleções · ~43 documentos de seed
- Consultas: 7 operações com \$and, \$or, \$gt/\$lt, \$exists, projeção, sort e paginação
- Aggregations: 5 pipelines com \$group, \$lookup, \$unwind, \$cond e aritmética
- Repositório: github.com/P3droGs/Mini-ABP-Bdn